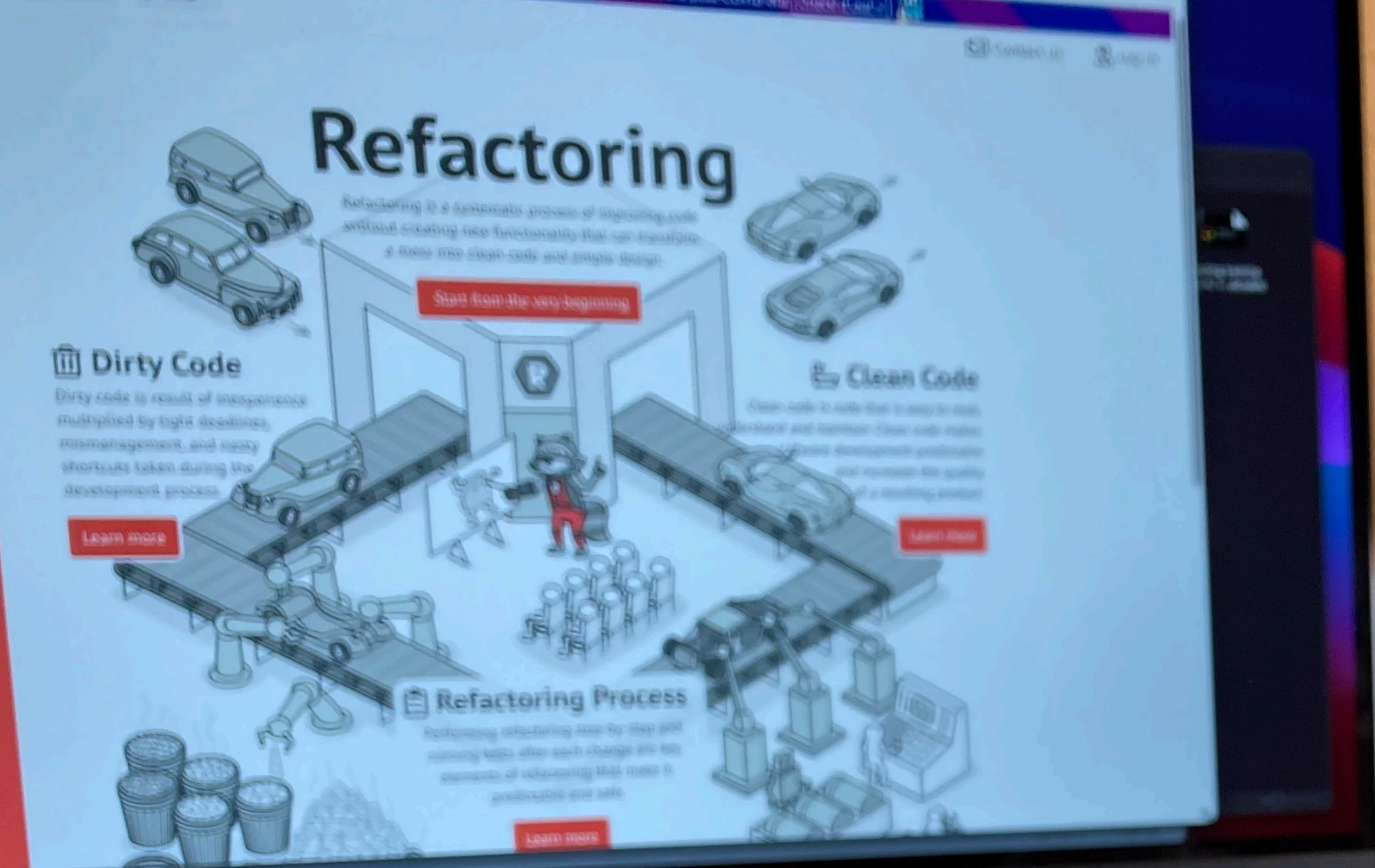


Refactoring Avanzado

por Fran Iglesias
Senior Software Engineer





Bloque 0

Introducción

Una triste historia para empezar...

Ejercicio

¿Qué hace este código?

```
class E {  
    private s: number;  
    private y: number;  
  
    constructor(s: number, y: number) {  
        this.s = s;  
        this.y = y;  
    }  
  
    bns(): number {  
        if (this.y > 5) {  
            return this.s * 0.2;  
        }  
        return this.s * 0.1;  
    }  
}
```


Ejercicio

¿Como introducirías el código para añadir un nuevo tipo de descuento (3 x 2)?

```
export function processOrdersWithDiscounts(
  orders: Array<{ id: string; items: Array<{ price: number }>; customer: { isVip: boolean } }>,
): number {
  let total = 0
  for (const order of orders) {
    if (order.items && order.items.length > 0) {
      for (const item of order.items) {
        if (order.customer && order.customer.isVip) {
          if (item.price > 100) {
            total += item.price * 0.8 // gran descuento VIP
          } else {
            total += item.price * 0.9 // pequeño descuento VIP
          }
        } else {
          if (item.price > 100) {
            total += item.price * 0.95 // gran descuento regular
          } else {
            total += item.price // sin descuento
          }
        }
      }
    }
  }
  return total
}
```


Repositorio de material y ejercicios



github.com/franiglesias/refactoring-avanzado

friglesias / refactoring-avanzado

Type / to search

CodeIssuesPull requestsActionsProjectsSecurity and qualityInsightsSettings

refactoring-avanzadoPublic

PinWatch 0Fork 0Star 1

main1 Branch0 Tags

Go to fileTAdd fileCode

franiglesiasImprove of some test21e85db · 20 hours ago16 Commits

csharp	More Links fixed	5 days ago
docs	Improve of some test	20 hours ago
go	Unify documentation	3 months ago
java	Unify documentation	3 months ago
php	Unify documentation	3 months ago
python	Unify documentation	3 months ago
typescript	Unify documentation	3 months ago
.gitignore	Initial commit	3 months ago
Curso de Refactoring Avanzado.pdf	Initial commit	3 months ago
README.md	Improve of some test	20 hours ago

README

About

multi-language version of the refactoring course

Readme

Activity

1 star

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

franiglesias Fran Iglesias

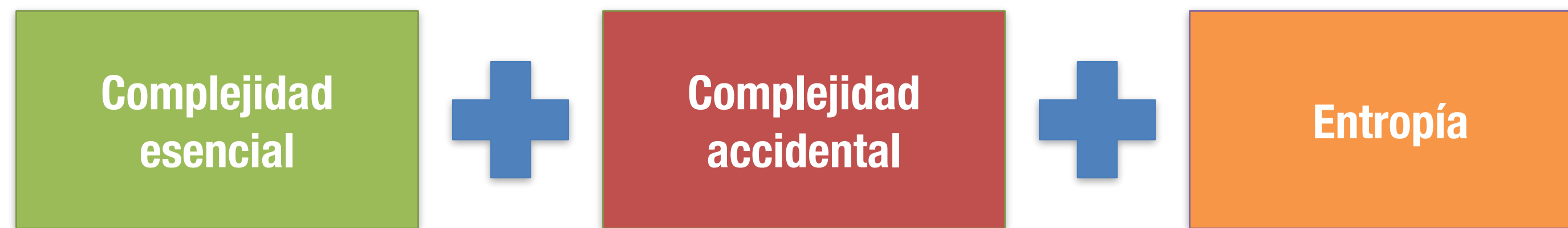
Languages

Bloque 1

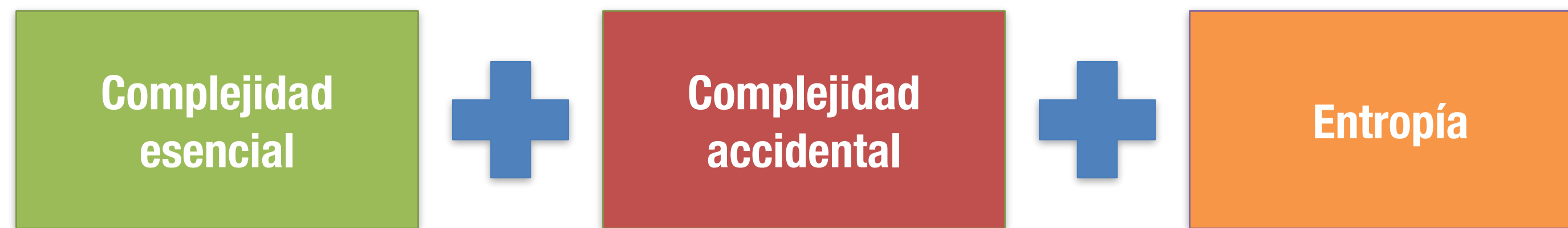
Economía del desarrollo de software

EL COSTE DEL CÓDIGO

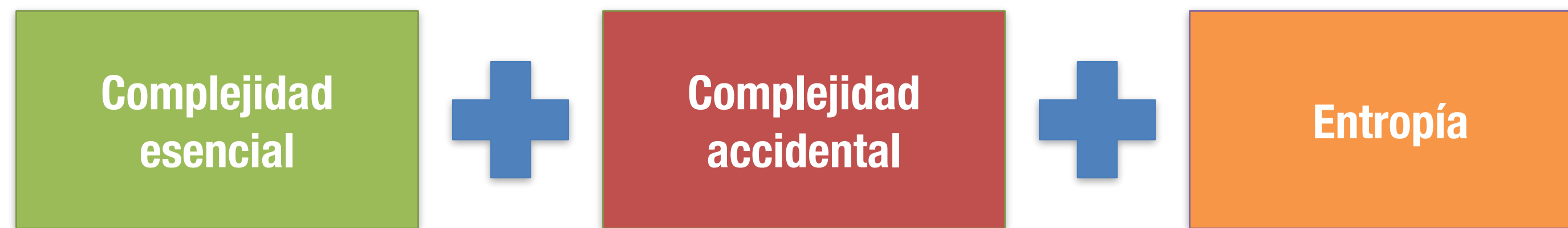




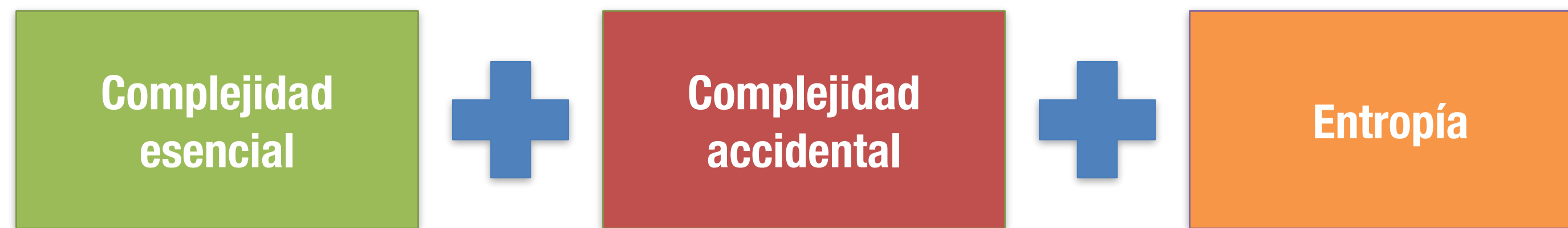
La propia del
problema que
resolvemos



Asociada con
las decisiones
técnicas con
las que se
implementa la
solución



El desorden del
código debido
a la
acumulación
de los cambios
en el tiempo

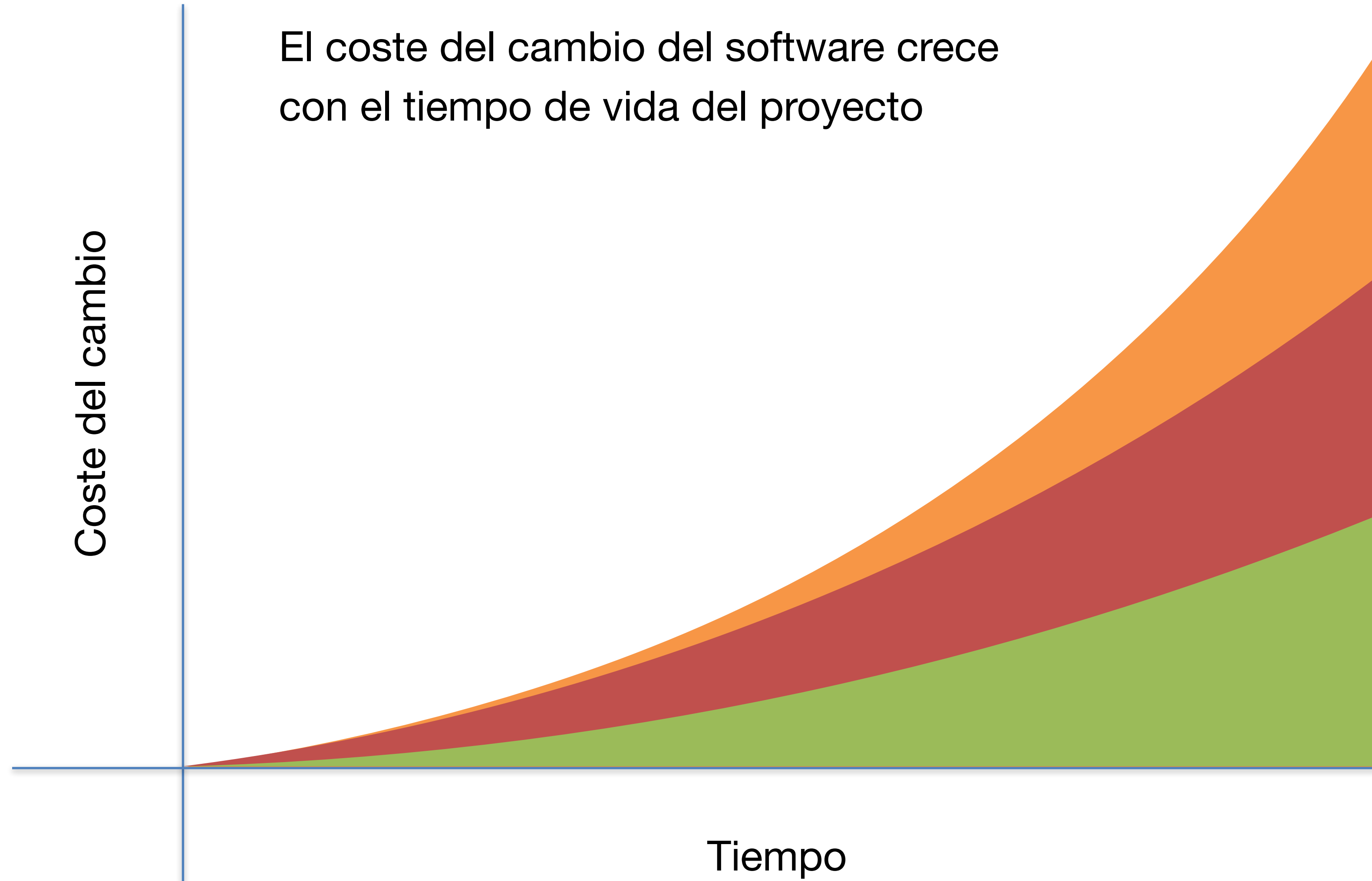


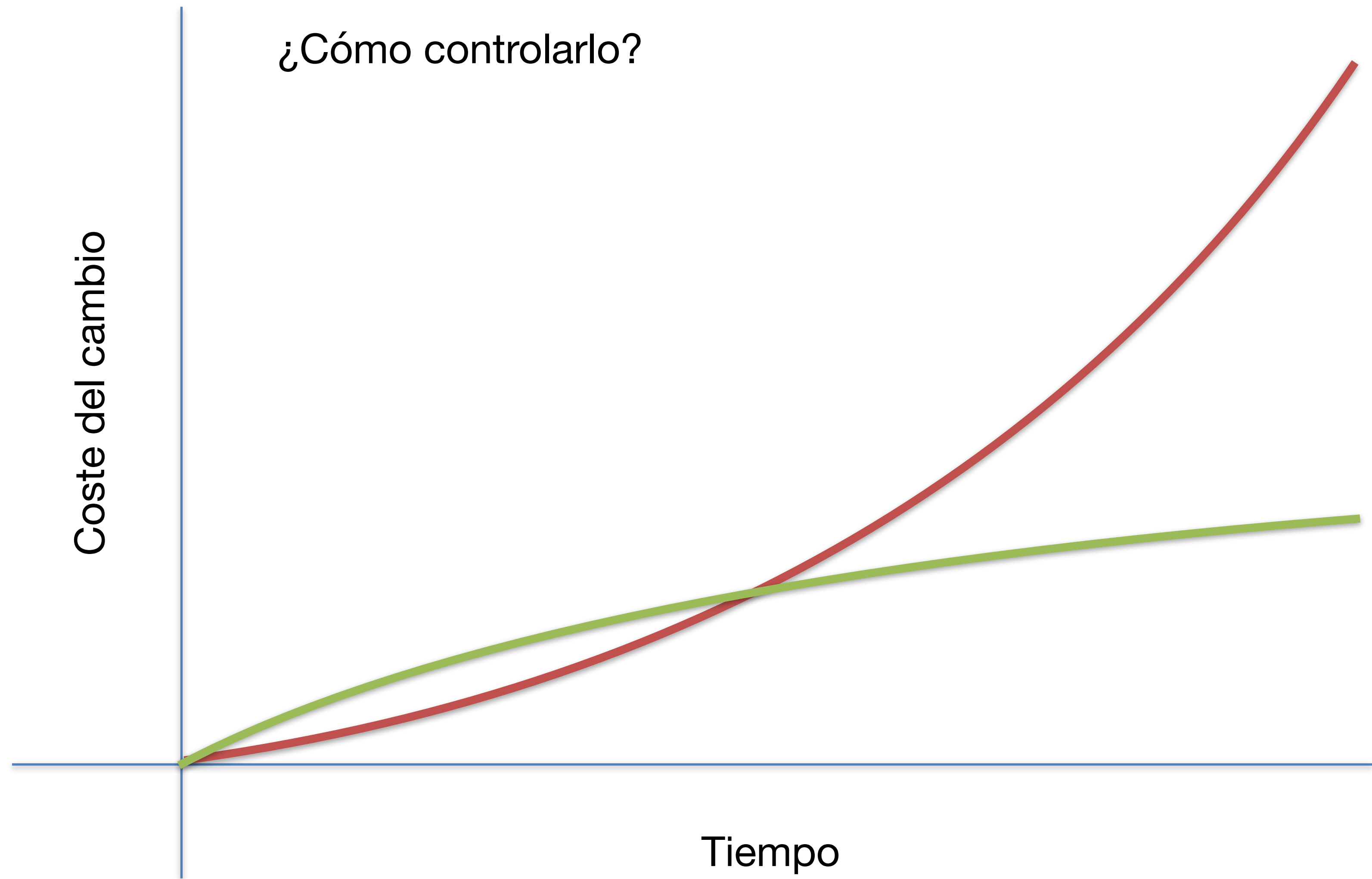
La propia del
problema que
resolvemos

Asociada con
las decisiones
técnicas con
las que se
implementa la
solución

El desorden del
código debido
a la
acumulación
de los cambios
en el tiempo

El coste del cambio del software crece
con el tiempo de vida del proyecto

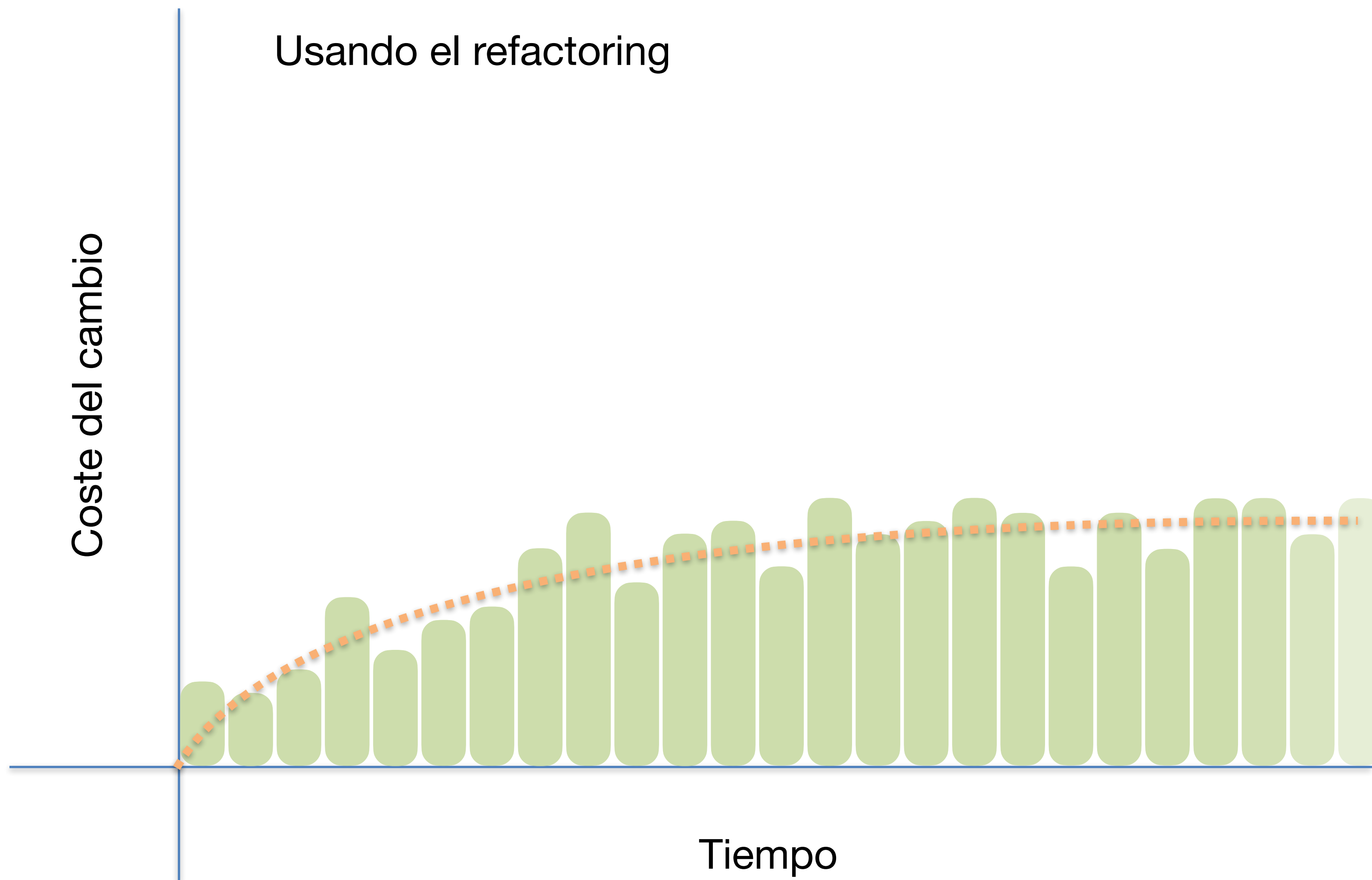




Usando el refactoring

Coste del cambio

Tiempo



El *refactoring* actúa sobre

**Complejidad
esencial**



**Complejidad
accidental**



Entropía

Mejorando el
modelado de la
solución

Reduciendo el
acoplamiento
con las
tecnologías de
tercera parte

Manteniendo
control sobre
los efectos de
la acumulación
de cambios

Ejercicio

¿A qué tipo de complejidad afectan estos requerimientos?

1. Introducir un nuevo estado en el ciclo de vida de un objeto de negocio.
2. Dar soporte a un nuevo método de pago.
3. Dar soporte a un nuevo requisito legal que obliga a tener una firma digital de una operación.
4. Cambiar el formato en que se generan ciertos informes.
5. Añadir un endpoint a una API para habilitar una nueva funcionalidad
6. Cambiar el ORM usado para hablar con una base de datos

**Complejidad
esencial**

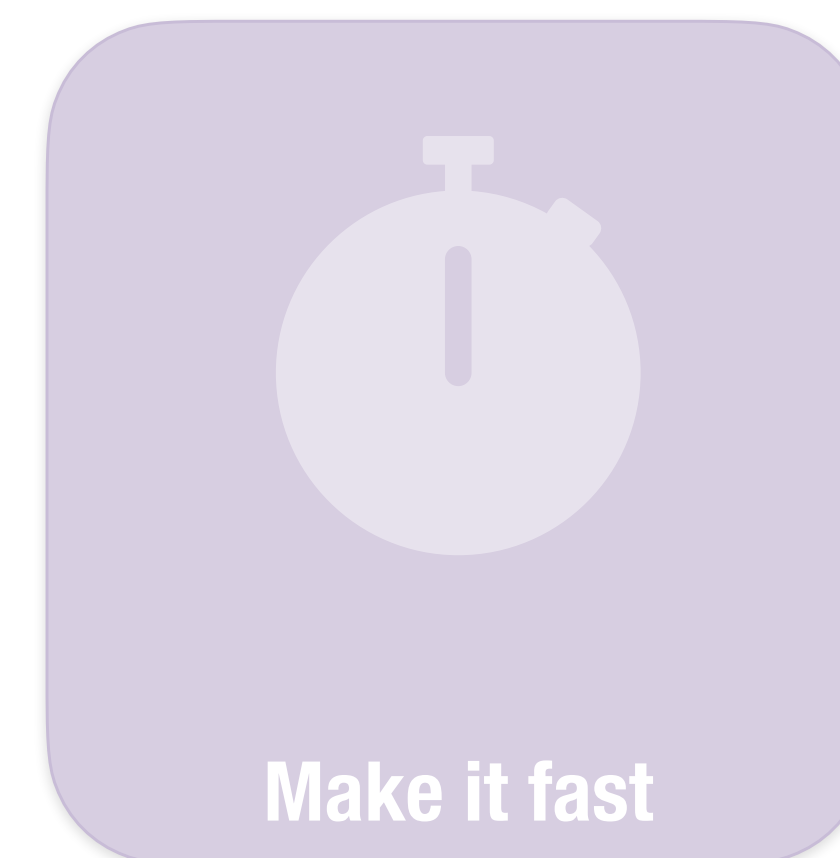
**Complejidad
accidental**

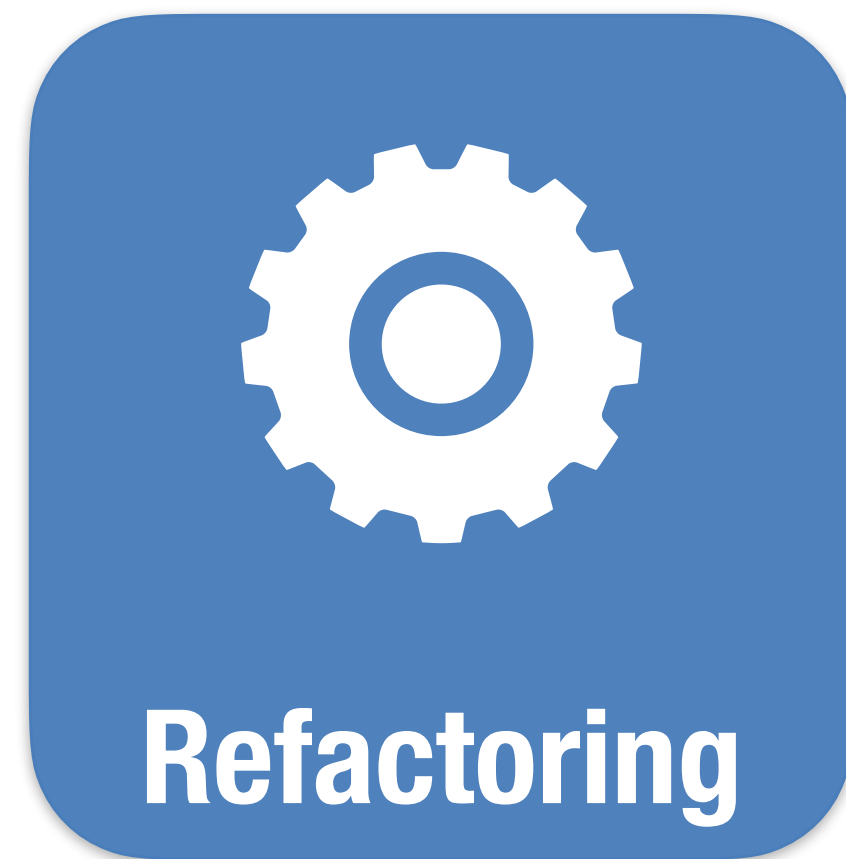
Entropía

QUÉ ES REFACTORING

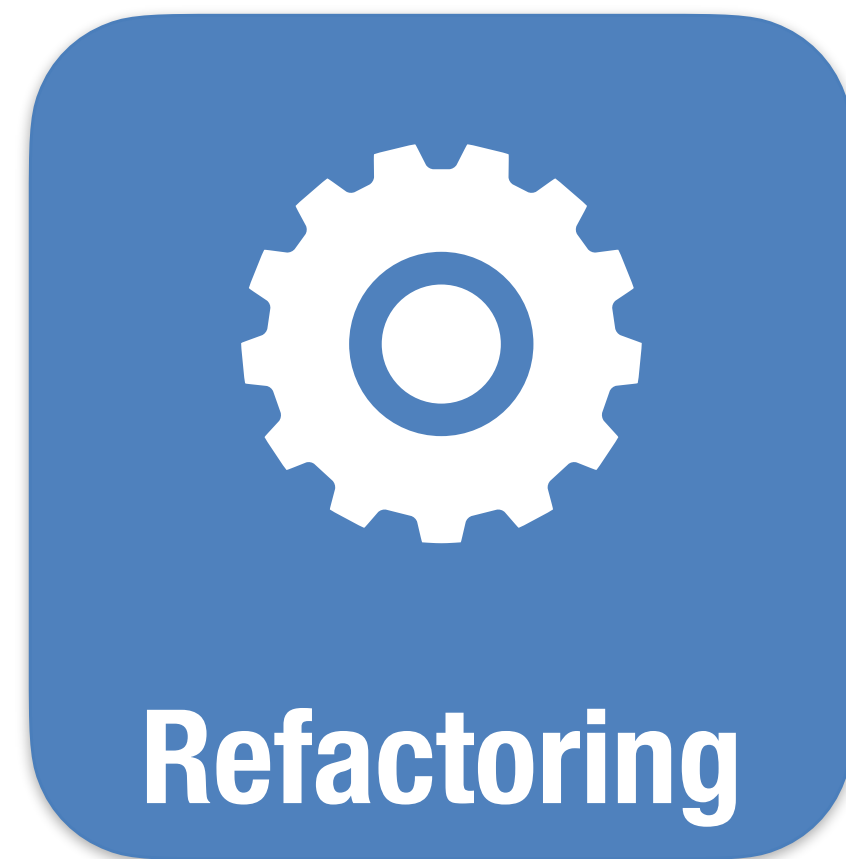


Kent Beck





Un proceso intencionado y sistemático por el que modificamos la estructura o diseño del código sin alterar su comportamiento a fin de conseguir que su mantenimiento sea sostenible en el largo plazo.



El refactoring es una herramienta económica: nos ayuda a mantener bajo control el coste del desarrollo de software.

QUÉ NO ES REFACTORING



Optimización

Puede incrementar la complejidad



Reescritura

El código actual, por problemático que sea paga las facturas

CUÁNDO REFACTORIZAR



En cualquier
momento en el
que
encontramos
un código que
no entendemos



Preparando el
código para
facilitar la
introducción de
una nueva
característica



Leyendo código



Refactor preparatorio



Refactor a posteriori

Tras introducir
un cambio, para
que sea fácil de
entender y
extender en el
futuro



En cualquier momento en el que nos encontramos un código que no entendemos



Preparando el código para facilitar la introducción de una nueva característica



Tras introducir un cambio, para que sea fácil de entender y extender en el futuro

*QUÉ QUEREMOS
CONSEGUIR*

4 reglas del diseño simple





Hace lo que dice

Previene efectos no deseados

Menos tiempo con Bugs



Dice lo que hace

Facilita entender cómo funciona

Menos tiempo lectura



Hay un sitio para cada unidad de conocimiento

Facilita intervenir

Menos tiempo intervención



Mantiene controlada complejidad

Facilita saber dónde intervenir

Más fácil intervenir



Hace lo que dice

Previene efectos no deseados

Menos tiempo con Bugs



Dice lo que hace

Facilita entender cómo funciona

Menos tiempo lectura



Hay un sitio para cada unidad de conocimiento

Facilita intervenir

Menos tiempo intervención



Mantiene controlada complejidad

Facilita saber dónde intervenir

Más fácil intervenir



Hace lo que dice

Previene efectos no deseados

Menos tiempo con Bugs



Dice lo que hace

Facilita entender cómo funciona

Menos tiempo lectura



Hay un sitio para cada unidad de conocimiento

Facilita intervenir

Menos tiempo intervención



Mantiene controlada complejidad

Facilita saber dónde intervenir

Más fácil intervenir



Hace lo que dice

Previene efectos no deseados

Menos tiempo con Bugs



Dice lo que hace

Facilita entender cómo funciona

Menos tiempo lectura



Hay un sitio para cada unidad de conocimiento

Facilita intervenir

Menos tiempo intervención



Mantiene controlada complejidad

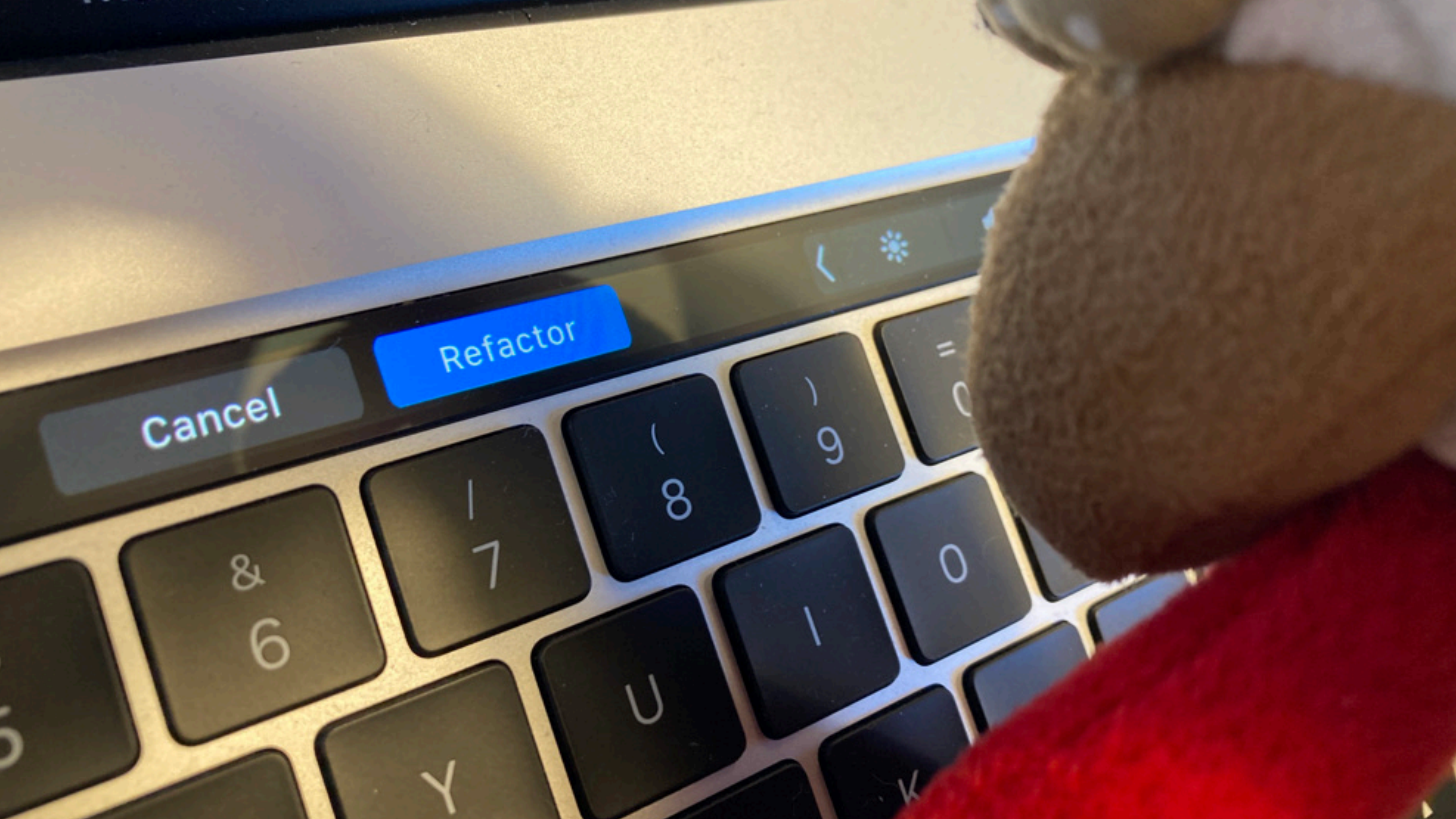
Facilita saber dónde intervenir

Más fácil intervenir

REFACTOR

Cancel

Refactor

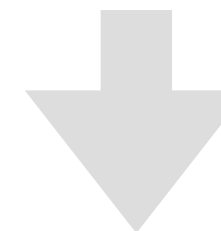


¿Qué es un refactor?

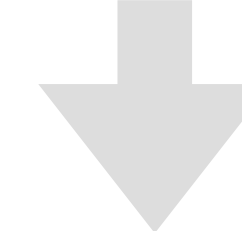
Es un procedimiento concreto para obtener una determinada transformación en el código

```
function tax(p: number): number {  
    return p * 1.21  
}
```

Rename



Rename



```
function applyVAT(price: number): number {  
    return price * 1.21  
}
```

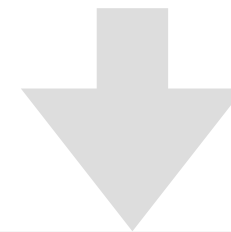
Refactor probado

Es un refactor cuyo efecto es predecible y habitualmente está automatizado.

Para aplicarlo no necesitamos la protección de tests.

```
function applyVAT(price: number): number {  
    return price * 1.21  
}
```

Introduce constant

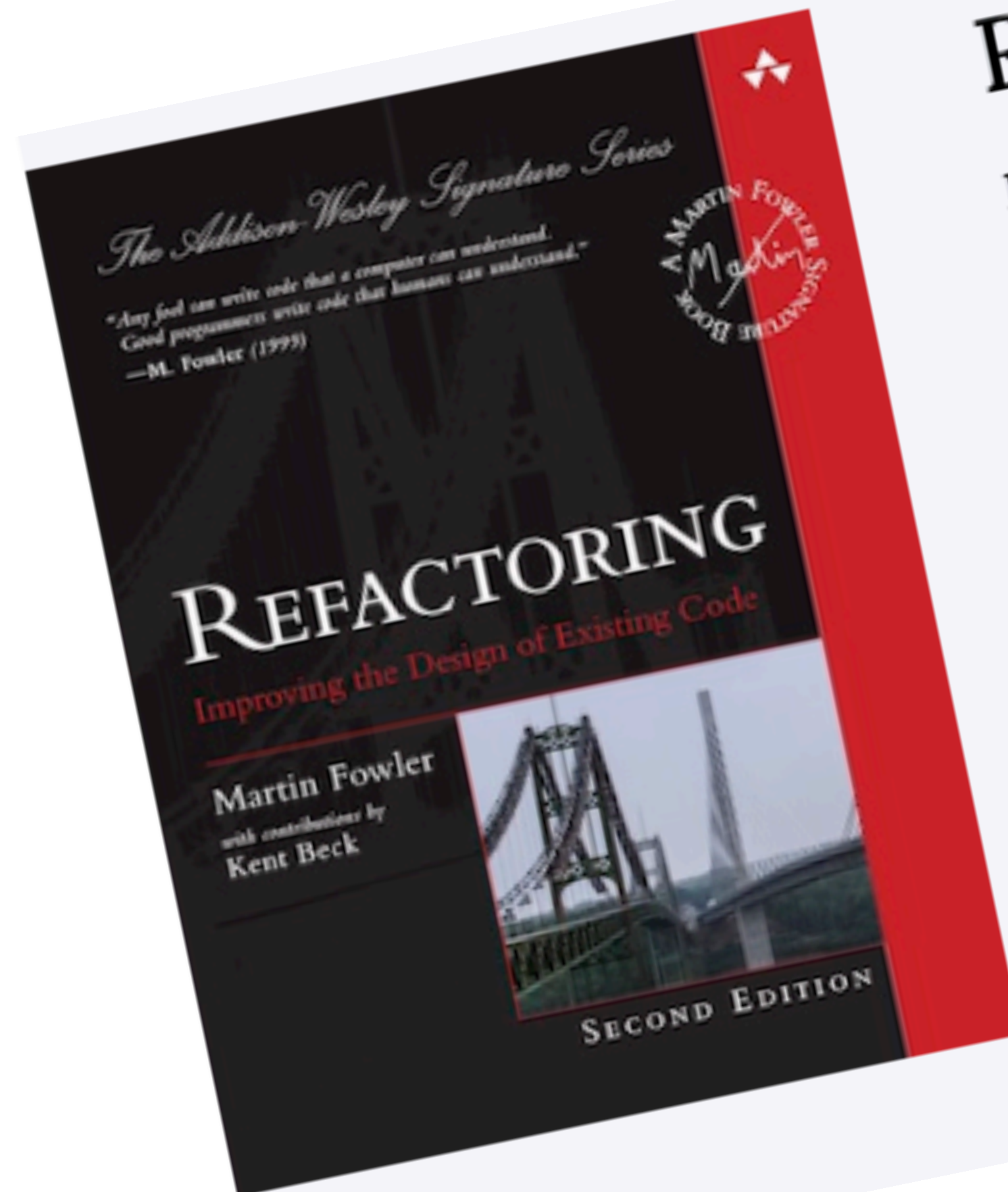


```
const VAT_RATE=1.21  
function applyVAT(price: number): number {  
    return price * VAT_RATE  
}
```


RECURSOS

El manual de referencia

<https://martinfowler.com/books/refactoring.html>



Refactoring

Improving the Design of Existing Code

by Martin Fowler, with Kent Beck
2018

The guide to how to transform code with safe and rapid process, vital to keeping it cheap and easy to modify for future needs.

amazon

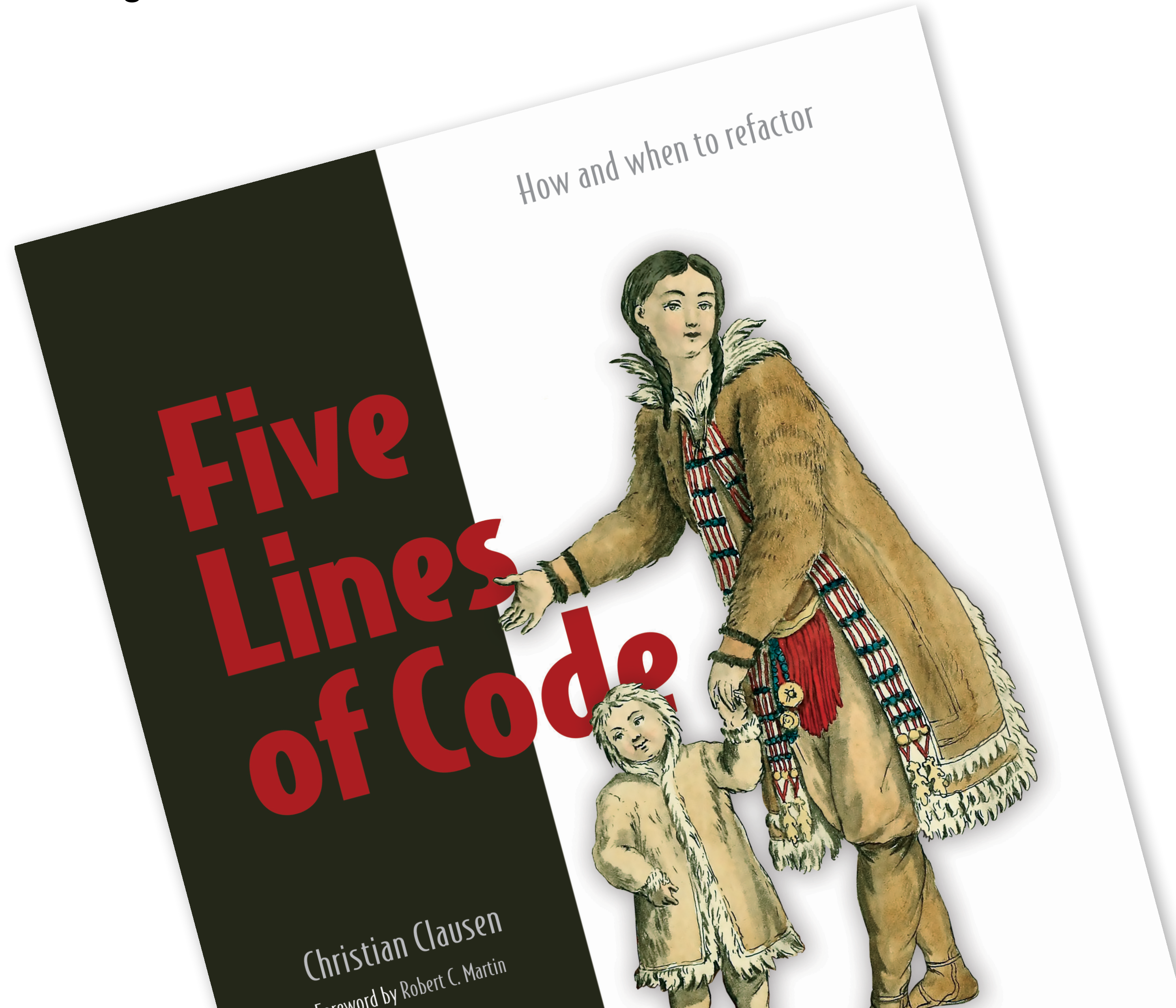
informIT

Notes for buying my books



Five Lines of Code

<https://www.manning.com/books/five-lines-of-code>



Software economics

<https://leanpub.com/software-economics>



Con el curso...

La guía del refactor cotidiano

Mantener tu código en forma es un trabajo para todos los días



Este libro está completo al 100%
ÚLTIMA ACTUALIZACIÓN EL 2025-01-11

\$7.99
PRECIO MÍNIMO

\$19.99
PRECIO SUGERIDO ?

USTED PAGA

\$19.99

EL AUTOR GANA

\$15.99

USTED PAGA

\$19.99

Cientes de la UE: Precio sin IVA.
El IVA se añade durante el pago.

Puede pagar en US \$ o en su moneda local (EUR, GBP, CAD, etc.)
cuando realice el pago con tarjeta de crédito usando Stripe.

Añadir Ebook al Carrito

[Añadir a la Lista de Deseos](#)

Pon tu código en forma con calistenia

Conoce, practica y aplica los consejos de Object Calisthenics, para mejorar la calidad del diseño de tu código



This book is 100% complete
LAST UPDATED ON 2023-11-29

\$19.00
MINIMUM PRICE

\$29.00
SUGGESTED PRICE ?

YOU PAY

\$29.00

AUTHOR EARNS

\$23.20

YOU PAY

\$29.00

EU customers: Price excludes VAT.
VAT is added during checkout.

You can pay in US \$ or in your local currency (EUR, GBP, CAD, etc.)
when you checkout with a credit card using Stripe.

Add Ebook to Cart

[Add to Wish List](#)





**REFACTORING
• GURU •**

- ★ Contenido premium
- 🔗 Patrones de diseño
- ✂ Refactorización *(próximamente)*
 - What is Refactoring
 - Catalog
 - Code Smells
 - Refactorings

☆⁺ Mi tienda Español

Refactoring

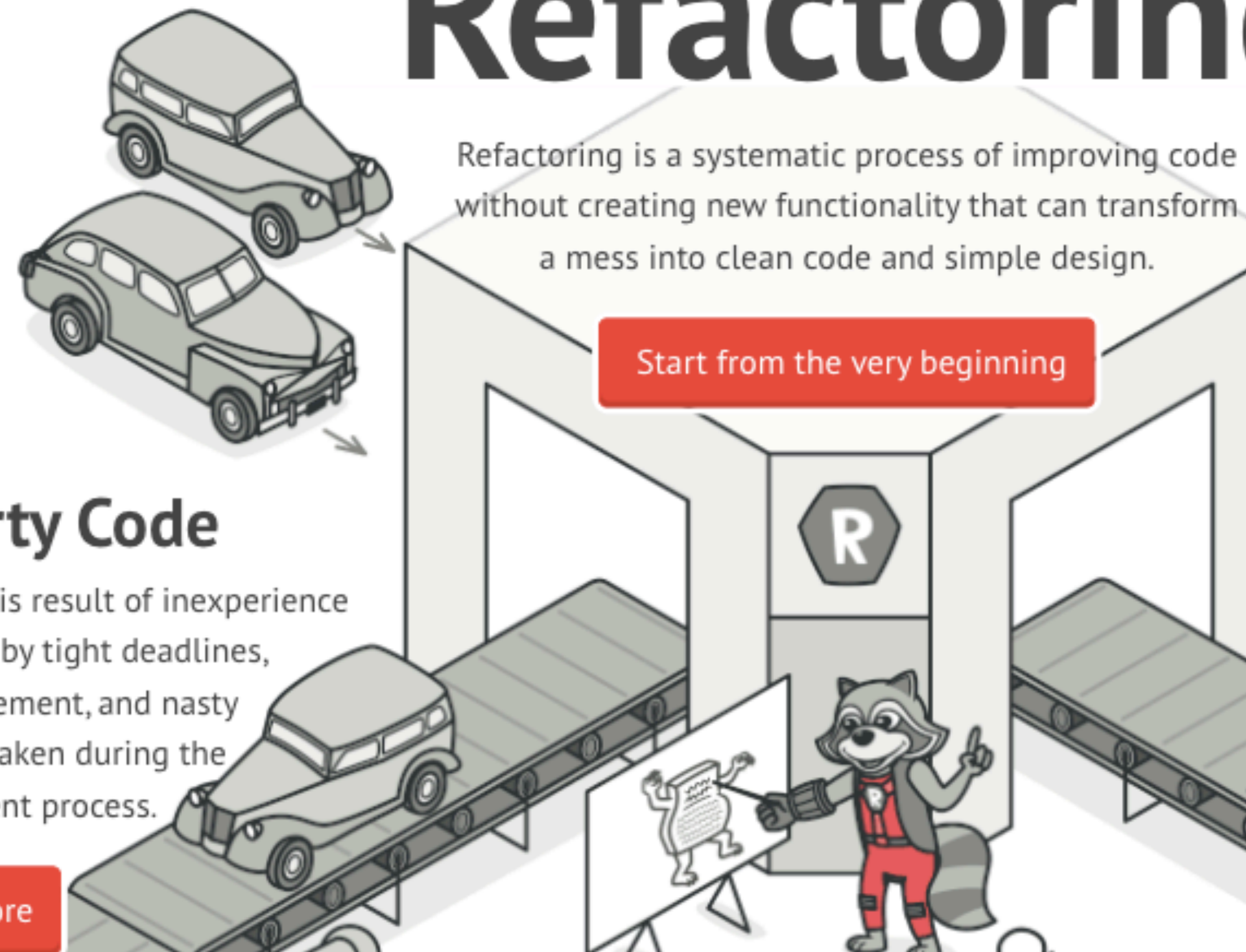
Refactoring is a systematic process of improving code without creating new functionality that can transform a mess into clean code and simple design.

Start from the very beginning

🗑 Dirty Code

Dirty code is result of inexperience multiplied by tight deadlines, mismanagement, and nasty shortcuts taken during the development process.

Learn more



Repositorio de material y ejercicios



github.com/franiglesias/refactoring-avanzado

Joppy - Matc...es you happy | Desarrollo | Prensa | Dev Quality Resources | franiglesias (Fran Iglesias) | Shop Stickers...ny Tee Shirts

Página principal | safari extension generate QR for url - Buscar con Google

franiglesias / refactoring-avanzado

Type / to search

Code | Issues | Pull requests | Actions | Projects | Security and quality | Insights | Settings

refactoring-avanzado Public

Pin | Watch 0 | Fork 0 | Star 1

main | 1 Branch | 0 Tags

Go to file | Add file | Code

franiglesias Improve of some test 21e85db · 20 hours ago 16 Commits

cssharp	More Links fixed	5 days ago
docs	Improve of some test	20 hours ago
go	Unify documentation	3 months ago
java	Unify documentation	3 months ago
php	Unify documentation	3 months ago
python	Unify documentation	3 months ago
typescript	Unify documentation	3 months ago
.gitignore	Initial commit	3 months ago
Curso de Refactoring Avanzado.pdf	Initial commit	3 months ago
README.md	Improve of some test	20 hours ago

README

About

multi-language version of the refactoring course

Readme | Activity | 1 star | 0 watching | 0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

franiglesias Fran Iglesias

Languages

Bloque 2

Controlando la entropía: Calistenia

Calistenia

Un conjunto de reglas que, aplicadas, nos llevan a un mejor diseño del código. Nos proporcionan heurísticas para valorar si un código está introduciendo **entropía** o **complejidad accidental**.

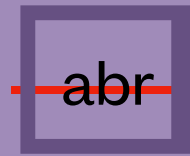


Reglas de Calistenia

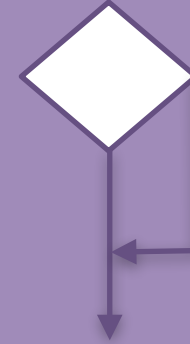
Se basan en fijarnos en ciertas características aplicables a cualquier código.

Si observamos que el código incumple alguna de esas reglas, intentamos arreglarlo para que se ajuste a ella.

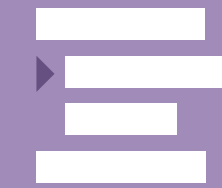
Reglas de calistenia



No usar
abreviaturas



No usar ELSE



Solo 1 nivel
indentación



Encapsular
primitivas



Colecciones de
Primera Clase



No getters/
setters



Unidades
pequeñas

2

Máx. 2
variables
instancia

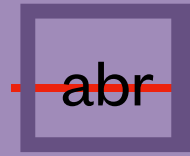


Un punto por
línea

Estas reglas se
pueden aplicar a
cualquier código, en
cualquier orden.

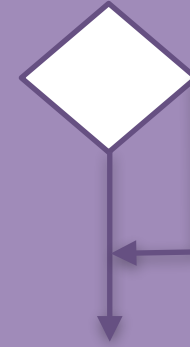
Si las tenemos
presentes al escribir
código o al
refactorizarlo, el
código irá
evolucionando
hacia un mejor
diseño.

Reglas de calistenia

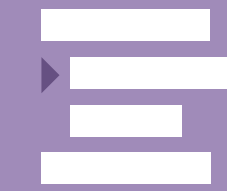


abr

No abreviaturas



No usar ELSE



Solo 1 nivel
indentación



123

Encapsular
primitivas



Colecciones de
Primera Clase



No getters/
setters



Unidades
pequeñas

2

Máy. 2
variables
instancia



Un punto por
línea

Tennis refactoring kata

<https://github.com/emilybache/Tennis-Refactoring-Kata>

1. Escoge una de las variantes de TennisGame
2. Identifica violaciones de las reglas de calistenia en esa variante
3. Modifica el código para reducir o eliminar esas violaciones



FizzBuzz Refactoring

<https://github.com/emilybache/FizzBuzz-Refactoring-Kata>

Queremos añadir las siguientes reglas:

7 o múltiplo de 7 -> Whizz

11 o múltiplo de 11 -> Bang

Refactorizar este código para que añadir la nueva función sea muy fácil. "Haz el cambio fácil, luego haz el cambio fácil".



Bloque 3

Code Smells

Patrones en el código que
revelan problemas en el
diseño



¿Qué es un *code smell*?



Un Code Smell es un patrón en el código que es el síntoma de un problema de diseño.



El código funciona correctamente, pero resulta difícil de entender o de cambiar.



Identificar code smells nos permite entender si un código necesita intervención y cuál.

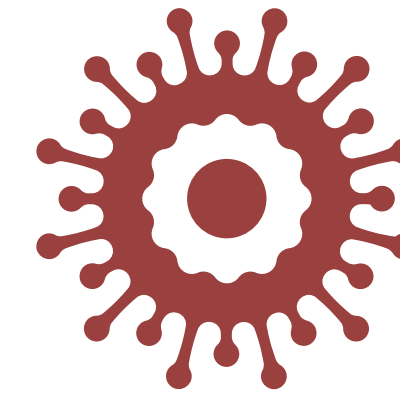
Categorías



Rebosantes



Desechables



Abusadores



Acopladores



Anti-cambios



Rebosantes

Unidades de código han crecido tanto que se hace difícil trabajar con ellas

Método Largo

Método con más de un cierto número de líneas no cohesivas (10)

Extraer métodos

Extraer clase

Clase Gigante

Clase con muchos métodos, que pueden ser largos, o muchas propiedades

Extraer clase

Agrupación de Datos

Datos que siempre van juntos y se utilizan juntos

Introducir Value Objects

Obsesión Primitiva

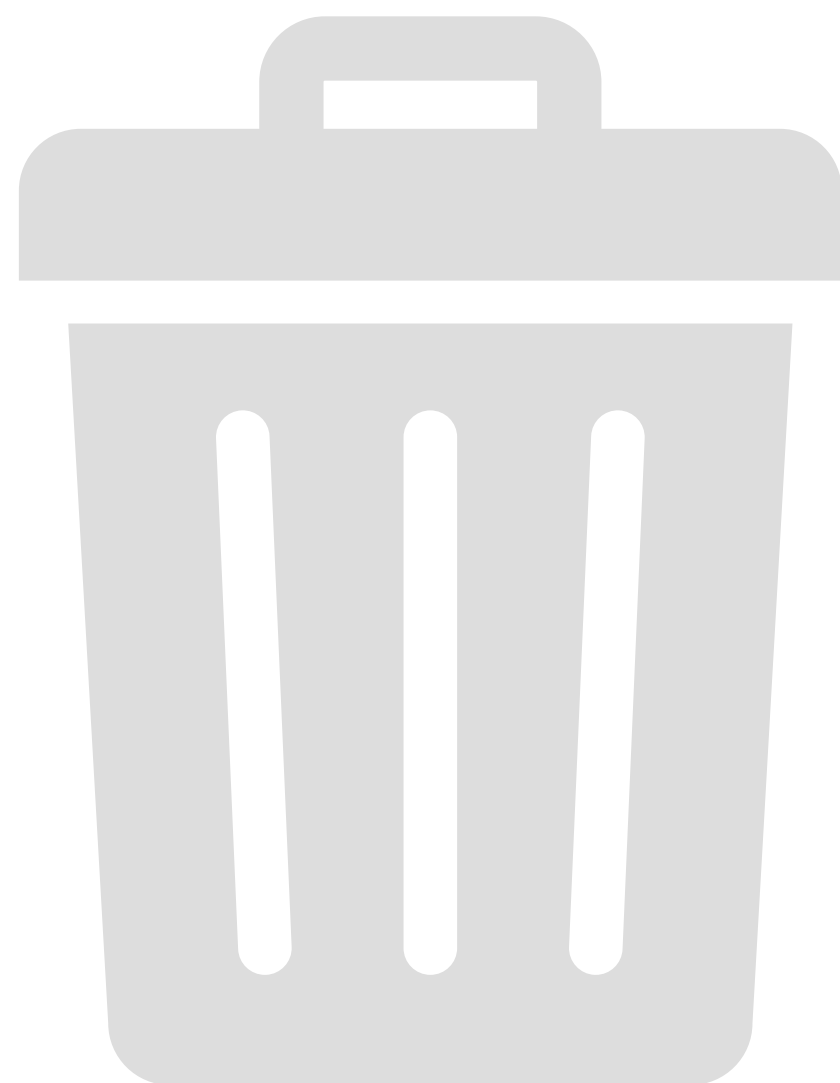
Representar conceptos con tipos primitivos en lugar de introducir propios

Introducir Value Objects

Lista Larga de Parámetros

Pasar más de 3 ó 4 parámetros a un método

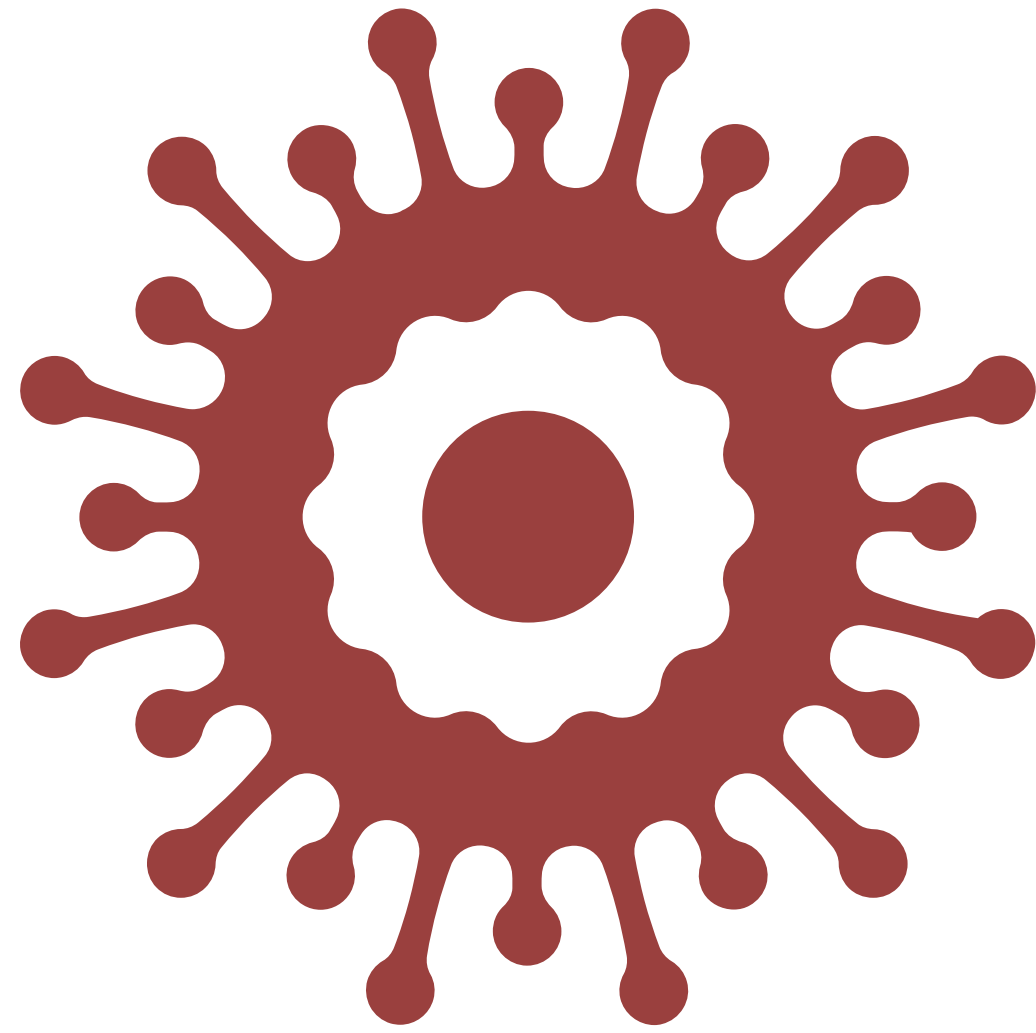
Introducir Objeto Parámetro



Desechables

Unidades de código que han perdido su razón de ser y sería mejor quitarlas.

Comentarios	Muchos comentarios que no aportan valor, incluyendo código comentado	Extraer métodos Quitar comentarios
Código duplicado	Fragmentos de código muy similares haciendo lo mismo en distintos lugares	Renombrar
Código muerto	Código que nunca se ejecuta	Borrar código
Clase perezosa	Clase que solo delega en otras, sin ninguna razón aparente	Borrar código
Clase de datos	Clase que no tiene comportamientos y solo aporta datos a otras	Extraer métodos Borrar código



Abusadores de la OOP

Consecuencia de aplicar de forma incorrecta los principios de la orientación a objetos

**Clases alternativas
dif. interfaz**

Dos clases que tienen un rol similar pero
cuya interfaz es diferente

**Introducir
Delegación**

Legado rechazado

Clase hija hereda métodos que no usa

**Introducir Interfaces
más estrechas**

Switch

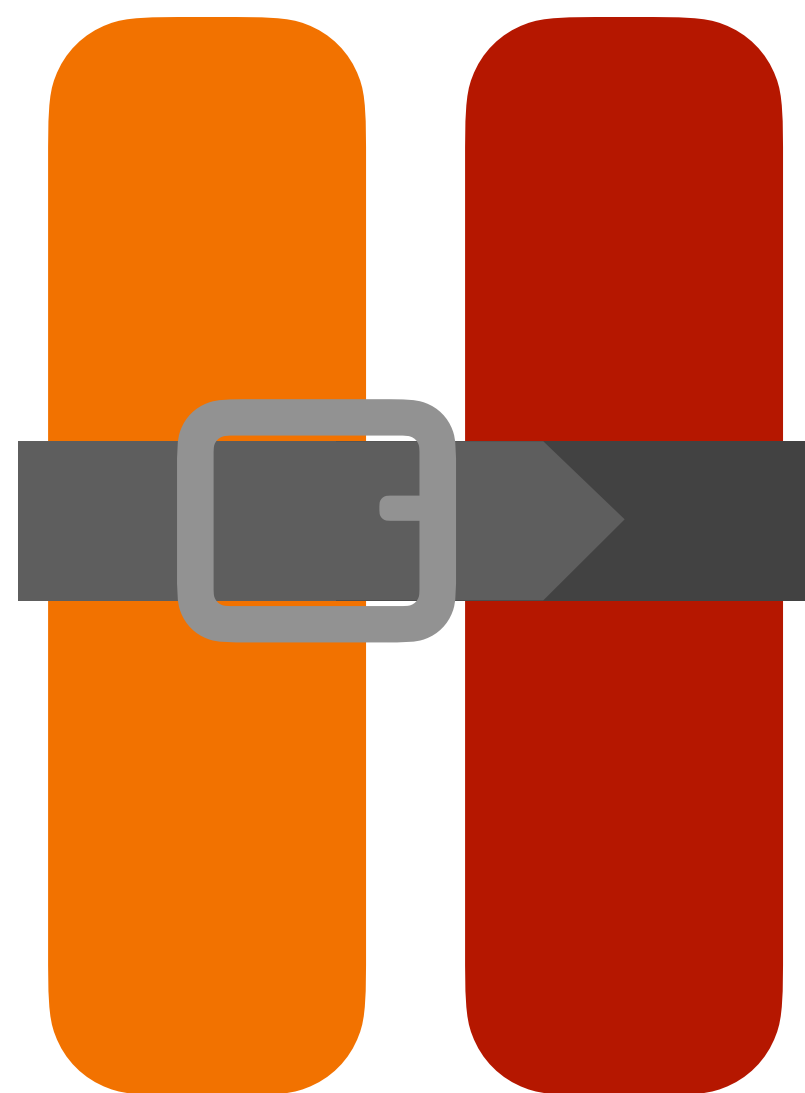
Usualmente un método que cambia su
comportamiento en función de una
propiedad

**Introducir
Polimorfismo**

Campo temporal

Solo se usa para llevar un valor a de un
método a otro

Borrar código



Acopladores

Cuando los cambios de un objeto afectan a otros

**Envidia de
características**

Un objeto usa más cosas de otro que de las suyas propias

Introducir Métodos

Mover a otra clase

**Intimidad
inapropiada**

Un objeto accede directamente a propiedades de otro

Introducir Métodos

**Cadenas de
mensajes**

Pedir algo a un objeto que está dentro de otro, que está dentro de otro, que está dentro de otro

Introducir Métodos

Ocultar delegación

**Middleman o
intermediario**

Hace una cosa, delegando en otra clase

**Eliminar
intermediario**



Anti cambios

Hacen difícil cambiar el código porque ha de hacerse en muchos lugares

Cambio divergente

Una clase tiene muchas razones para cambiar

Introducir clase

Jerarquía paralela

Si hago un cambio en una jerarquía también tengo que hacerlo en la otra

Introducir interfaz

Delegación

Cirugía a tiros

Para aplicar un cambio tengo que hacerlo en muchos sitios del código

Introducir clase

Variable de instancia temporal

Una propiedad que solo tiene valores para ciertas etapas del ciclo de vida

Introducir state pattern

Smelly Tic-Tac-Toe

<https://github.com/exeal-es/smelly-tic-tac-toe-kata>

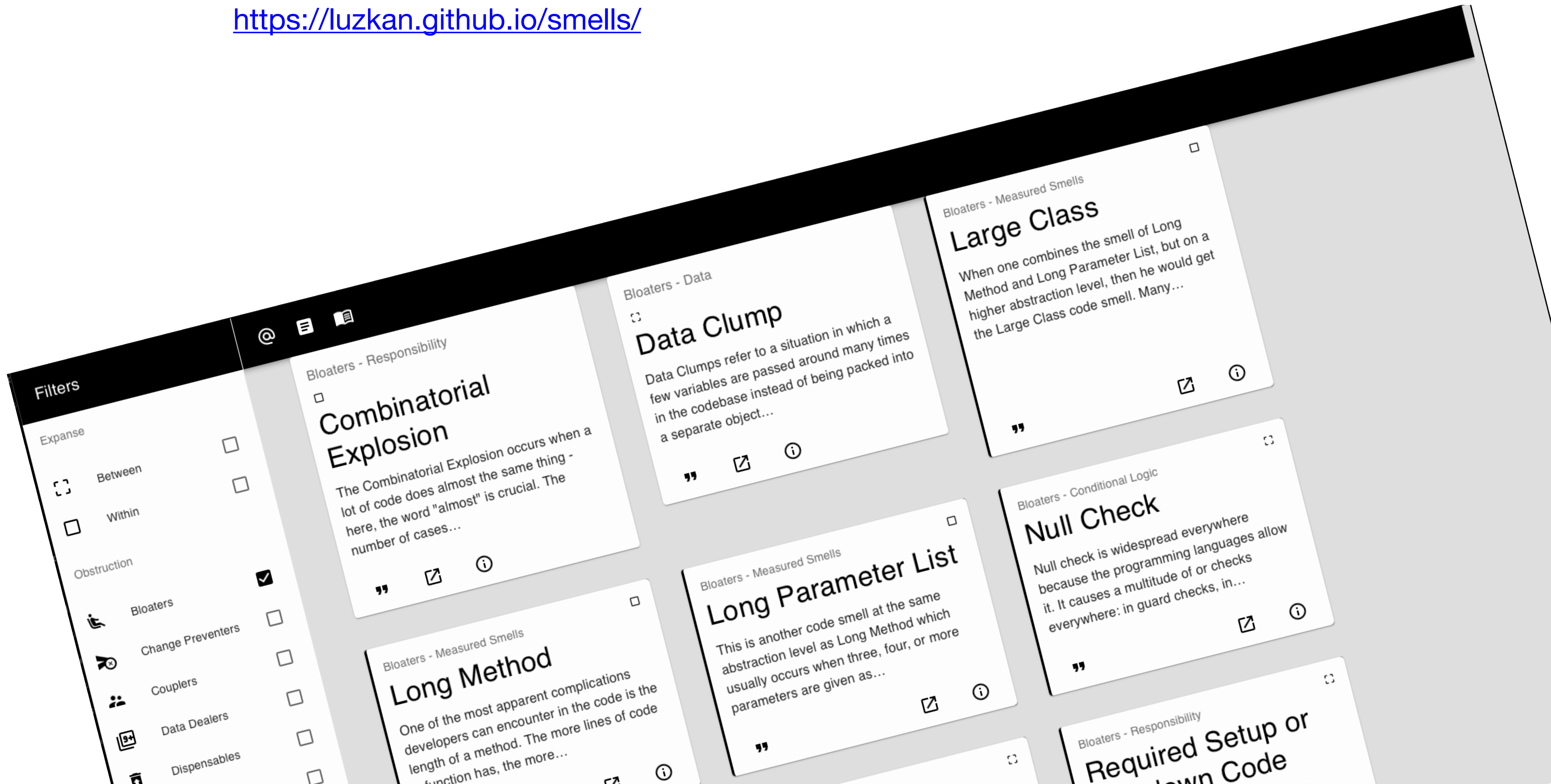
1. Identifica los code smells
2. Aplica refactors para resolverlos

- Primitive obsession
- Feature envy
- Data class
- Message chain
- Long method
- Comments
- Long parameter list
- Shotgun surgery
- Duplicated code
- Large class
- Divergent change
- Data clump
- Lazy class
- Dead code



Code smell catalog

<https://luzkan.github.io/smells/>



Bloque 4

Técnicas avanzadas

CAMBIO PARALELO

Expandir, migrar, contraer

Expandir

Migrar

Contraer

Expandir, migrar, contraer

Expandir

Escribe código
nuevo en lugar de
cambiar el que
existe.
Añade tests o
aplica TDD

Migrar

Contraer

Expandir, migrar, contraer

Expandir

Migrar

Contraer

Deprecia el
código a
reemplazar.
Empieza a usar el
código nuevo
siempre que
tengas
oportunidad.

Expandir, migrar, contraer

Expandir

Migrar

Contraer

Una vez que todo el código ha sido migrado, elimina el código deprecado, sus tests y otros restos

Expandir, migrar, contraer

Expandir

Escribe código nuevo en lugar de cambiar el que existe.
Añade tests o aplica TDD

Migrar

Depreca el código a reemplazar.
Empieza a usar el código nuevo siempre que tengas oportunidad.

Contraer

Una vez que todo el código ha sido migrado, elimina el código deprecado, sus tests y otros restos

Demo

<https://github.com/unclejamal/parallel-change>

Usando Parallel Change, modifica la clase ShoppingCart para poder manejar varios elementos en lugar de uno solo.

Los tests existentes tienen que pasar siempre.

Escribe nuevos tests si lo consideras necesario.



Ejercicio

Queremos hacer que User de soporte a nombre y apellidos

```
export class User {
  constructor(
    public readonly id: string,
    public readonly name: string,
    public readonly email: string,
  ) {}
}

// --- Consumers of User.name ---

export function formatGreeting(user: User): string {
  return `Hello, ${user.name}!`
}

export function formatEmailHeader(user: User): string {
  return `From: ${user.name} <${user.email}>`
}

export function formatDisplayName(user: User): string {
  return `${user.name} (${user.id})`
}

export function buildUserSummary(users: User[]): string {
  return users.map((u) => `- ${u.name}`).join('\n')
}
```

TESTS DE CARACTERIZACIÓN

Tests de Caracterización

Son los tests que escribimos para describir el comportamiento de código que ya existe, pero que no entendemos.

Estos tests tienen el objetivo de describir o caracterizar el comportamiento actual del código. De este modo, podemos refactorizarlo con seguridad.

Golden Master

Es una técnica en la que creamos un gran número de tests que prueban combinaciones de valores de parámetros que el código acepte.

Capturamos el resultado y lo guardamos para usarlo como criterio durante el proceso de refactor.

Ejercicio

```
export class ReceiptPrinter {
  // Do not change this function at the beginning of the exercise; first create the Golden Master.
  print(order: Order): string {
    const now = new Date(Date.now())

    const header = `Recibo ${order.id} - ${now.toLocaleDateString()} ${now.toLocaleTimeString()}`

    let subtotal = 0
    let lines = order.items.map((it, idx) => {
      const lineTotal = round(it.unitPrice * it.quantity)
      subtotal = round(subtotal + lineTotal)
      return `${idx + 1}. ${it.description} (${it.sku}) x${it.quantity} = ${lineTotal.toFixed(2)}`
    })

    let luckyDiscountPct = 0
    if (Math.random() < 0.1) {
      luckyDiscountPct = Math.random() * 0.05
    }
    const luckyDiscount = round(subtotal * luckyDiscountPct)

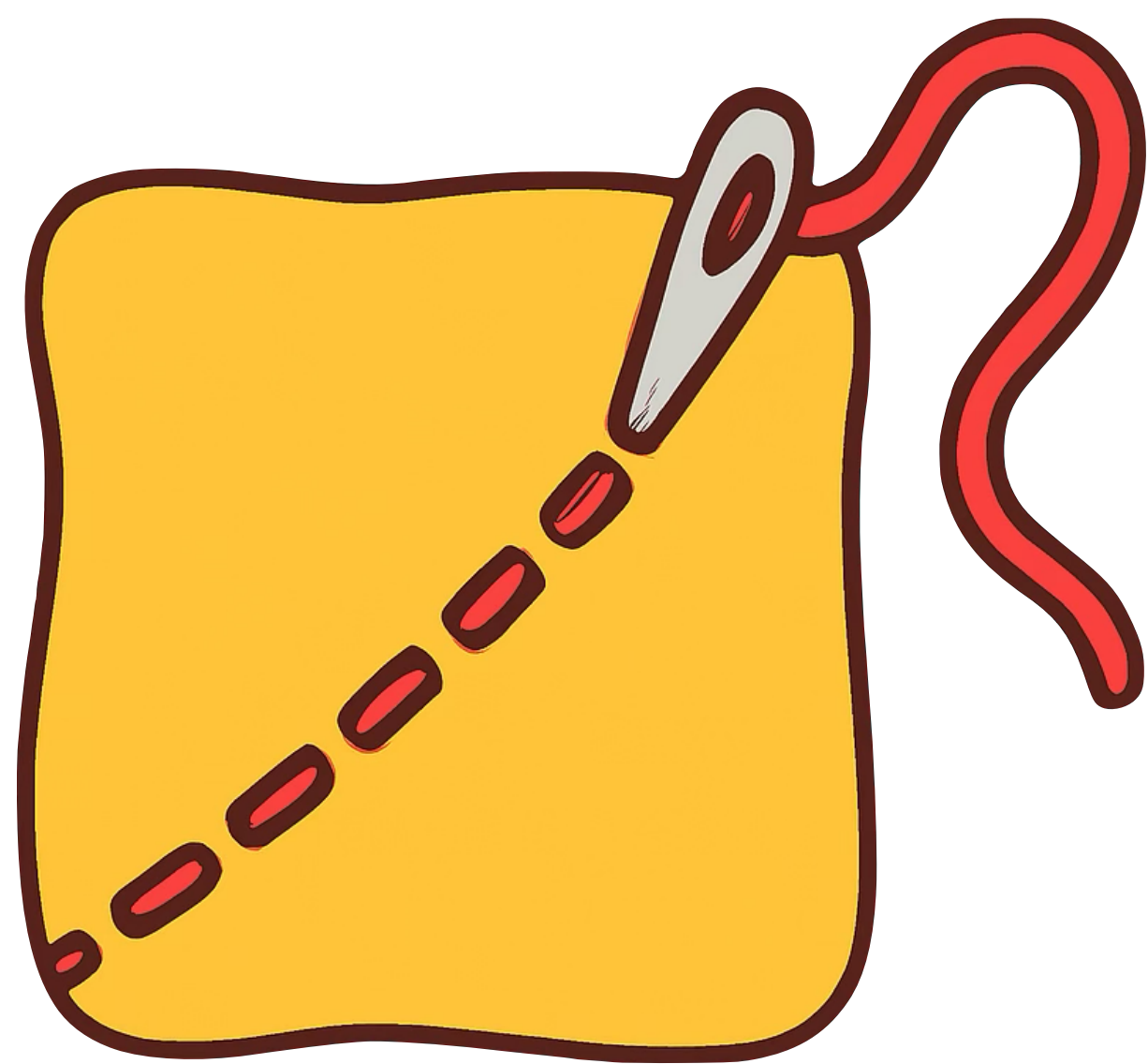
    const taxableGeneral = order.items
      .filter((i) => i.category !== 'books')
      .reduce((s, i) => s + (i.category === 'food' ? 0 : i.unitPrice * i.quantity), 0)
    const foodTax = order.items
      .filter((i) => i.category === 'food')
      .reduce((s, i) => s + i.unitPrice * i.quantity * 0.03, 0)
    const generalTax = taxableGeneral * 0.07
    const taxes = round(generalTax + foodTax)

    const total = round(subtotal - luckyDiscount + taxes)

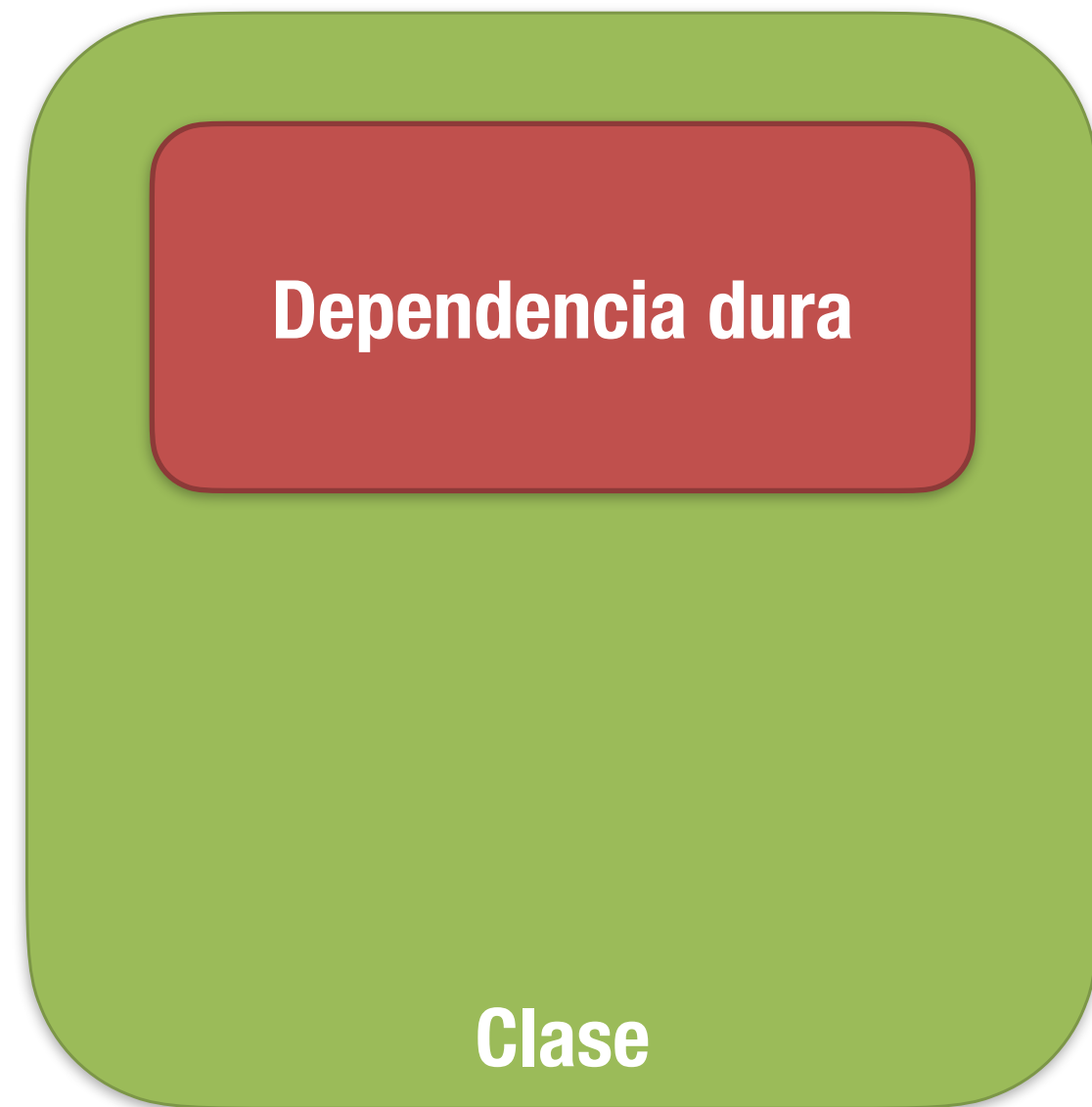
    const summary = [
      `Subtotal: ${subtotal.toFixed(2)}`,
      luckyDiscount > 0
        ? `Descuento de la suerte: -${luckyDiscount.toFixed(2)} (${(luckyDiscountPct * 100).toFixed(2)}%)`
        : `Descuento de la suerte: $0.00 (0.00%)`,
      `Impuestos: ${taxes.toFixed(2)}`,
      `TOTAL: ${total.toFixed(2)}`,
    ]

    return [header, ...lines, '---', ...summary].join('\n')
  }
}
```

ROMPER DEPENDENCIAS
DURAS

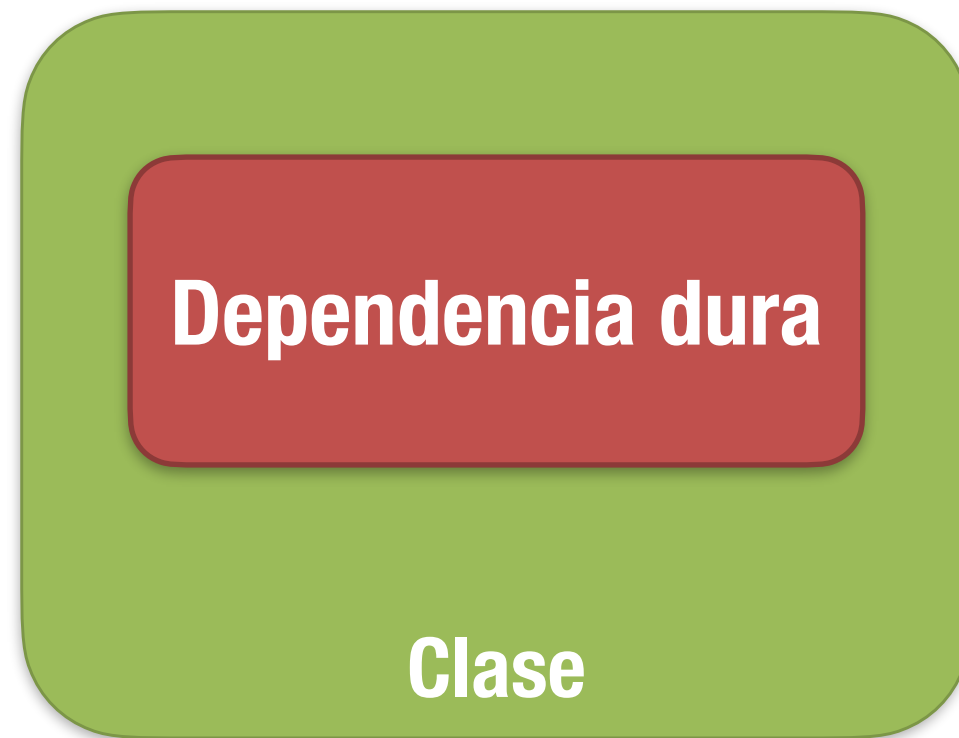


Seams o Costuras

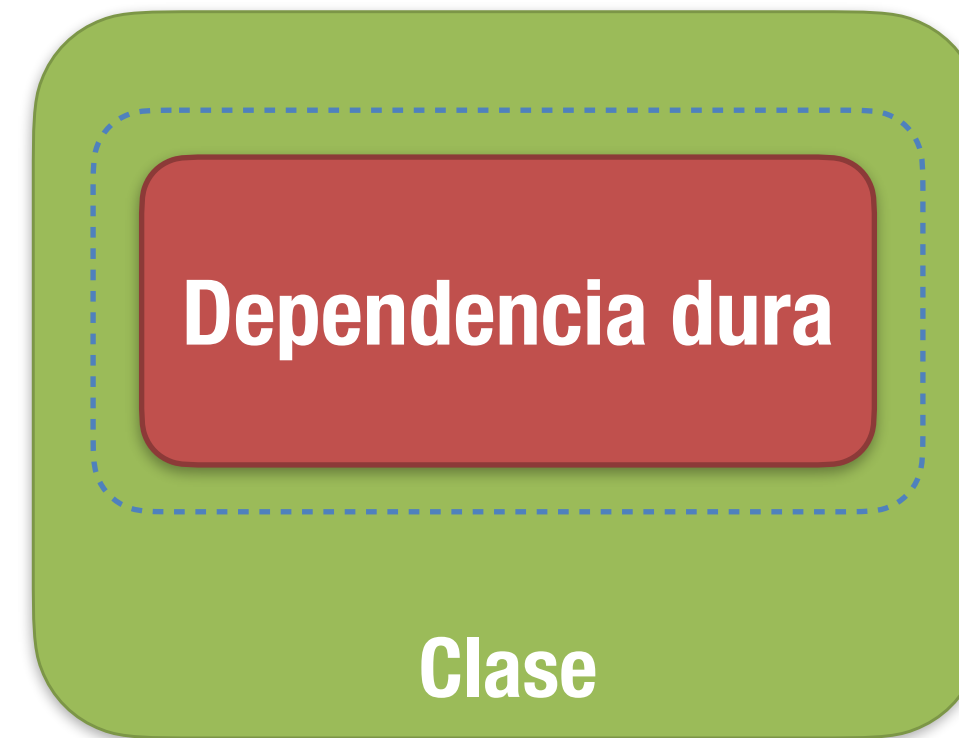


Una **dependencia dura** es un fragmento de código que impide que una clase se pueda poner bajo test.

Un ***seam o costura*** es un lugar en el código que nos permite cambiar el código de una manera controlada para poder introducir tests.



Identificar las dependencias que hacen la clase no testable



Seam: Aislar ese fragmento de código en un método *protected*



Extender la clase sobre escribiendo el seam
Usar esta Subclase en los tests
Invertir la dependencia

Ejercicio

```
export class ReceiptPrinter {
  // Do not change this function at the beginning of the exercise; first create the Golden Master.
  print(order: Order): string {
    const now = new Date(Date.now())

    const header = `Recibo ${order.id} - ${now.toLocaleDateString()} ${now.toLocaleTimeString()}`

    let subtotal = 0
    let lines = order.items.map((it, idx) => {
      const lineTotal = round(it.unitPrice * it.quantity)
      subtotal = round(subtotal + lineTotal)
      return `${idx + 1}. ${it.description} (${it.sku}) x${it.quantity} = ${lineTotal.toFixed(2)}`
    })

    let luckyDiscountPct = 0
    if (Math.random() < 0.1) {
      luckyDiscountPct = Math.random() * 0.05
    }
    const luckyDiscount = round(subtotal * luckyDiscountPct)

    const taxableGeneral = order.items
      .filter((i) => i.category !== 'books')
      .reduce((s, i) => s + (i.category === 'food' ? 0 : i.unitPrice * i.quantity), 0)
    const foodTax = order.items
      .filter((i) => i.category === 'food')
      .reduce((s, i) => s + i.unitPrice * i.quantity * 0.03, 0)
    const generalTax = taxableGeneral * 0.07
    const taxes = round(generalTax + foodTax)

    const total = round(subtotal - luckyDiscount + taxes)

    const summary = [
      `Subtotal: ${subtotal.toFixed(2)}`,
      luckyDiscount > 0
        ? `Descuento de la suerte: -${luckyDiscount.toFixed(2)} (${(luckyDiscountPct * 100).toFixed(2)}%)`
        : `Descuento de la suerte: $0.00 (0.00%)`,
      `Impuestos: ${taxes.toFixed(2)}`,
      `TOTAL: ${total.toFixed(2)}`,
    ]

    return [header, ...lines, '---', ...summary].join('\n')
  }
}
```


SPROUT



Sprout o Injerto

Es una técnica para introducir código nuevo (bien construido y bien testado) en una pieza de software legacy, reemplazando la existente, pero manteniendo su uso hasta tener la seguridad de que podemos hacer el cambio.

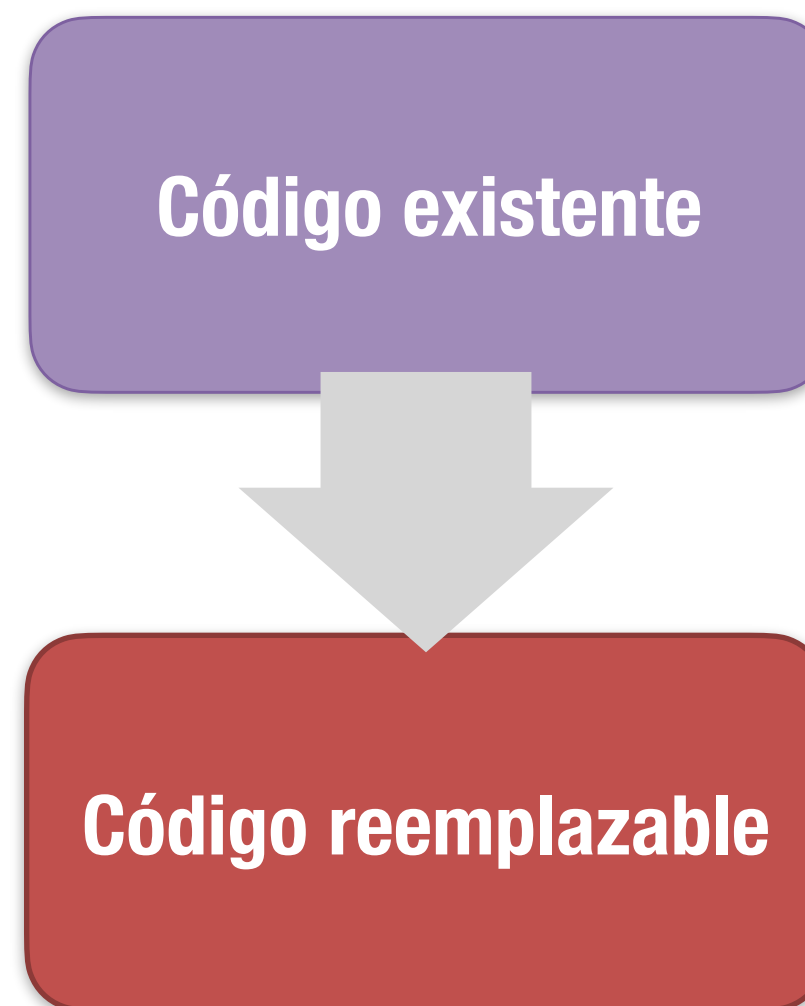
Aplica a situaciones en las que queremos reemplazar un algoritmo o una dependencia completamente.

Sprout / Injerto

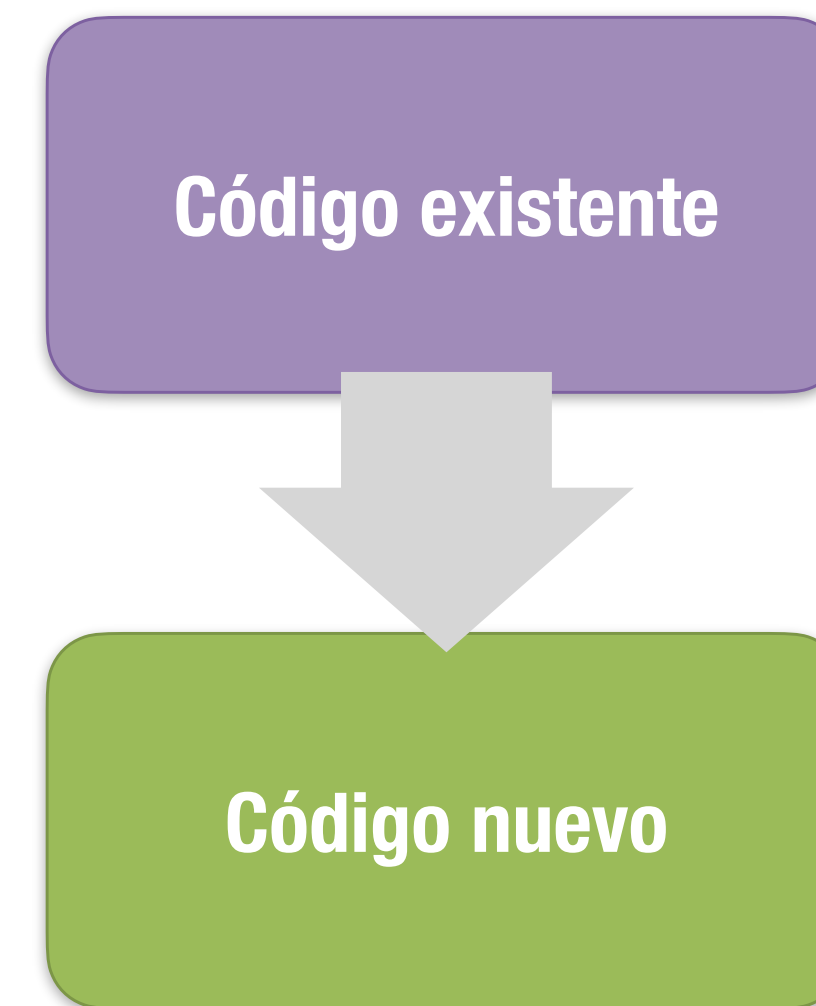
Escribe código nuevo que haga lo que necesitas.
Ponlo bajo test o desarróllalo con TDD.



Identifica desde donde deberías llamarlo en el código existente



Introduce las llamadas al nuevo código para reemplazar los usos del viejo



Ejercicio

Queremos introducir soporte de impuestos para otras regiones.

```
export function calculateTotal(cart: CartItem[], region: Region): number
{
  const subtotal = cart.reduce((s, it) => s + it.price * it.qty, 0)

  let tax = 0
  if (region === 'US') {
    tax = subtotal * 0.07 // 7% plano
  } else if (region === 'EU') {
    // exenciones ingenuas en línea
    const taxable = cart
      .filter((it) => it.category !== 'books' && it.category !== 'food')
      .reduce((s, it) => s + it.price * it.qty, 0)
    tax = taxable * 0.2 // 20% plano solo sobre los ítems gravables
  }

  return roundCurrency(subtotal + tax)
}
```


WRAP



Wrap o Envoltorio

Esta técnica consiste en aislar un código problemático, de forma que podamos mejorar o incluso reemplazar su uso, pero sin perder la interfaz.

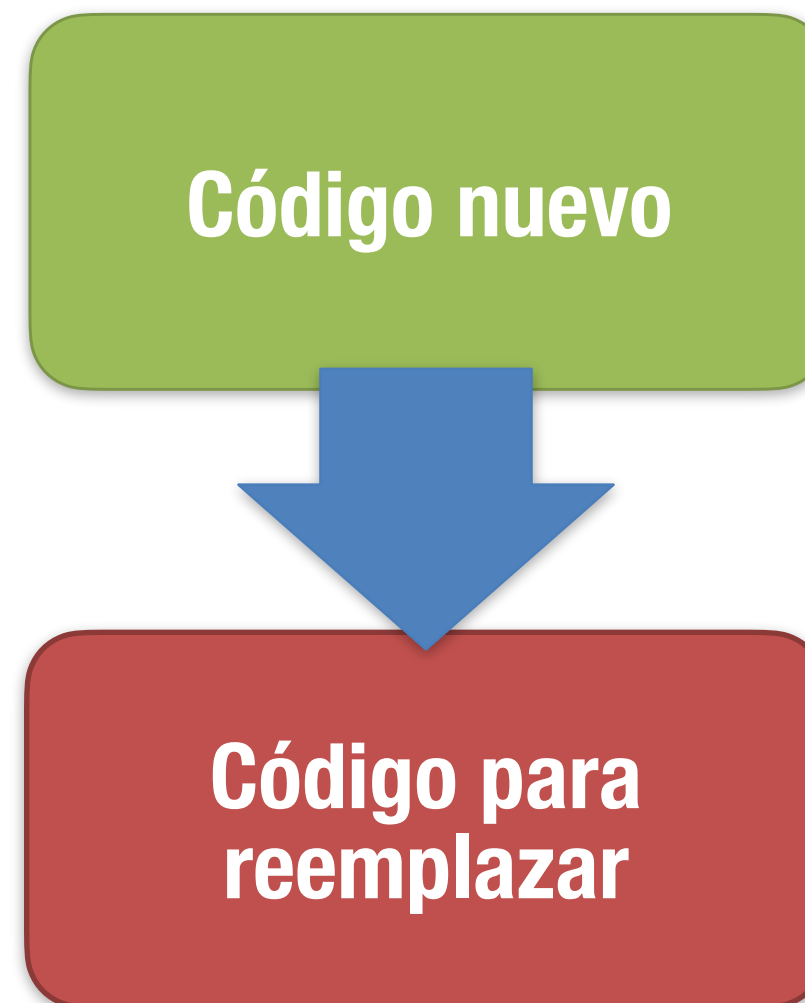
Aplica cuando queremos mejorar la funcionalidad de un código, pero no perder la compatibilidad con el uso que ya tenemos.

Wrap / Envoltorio

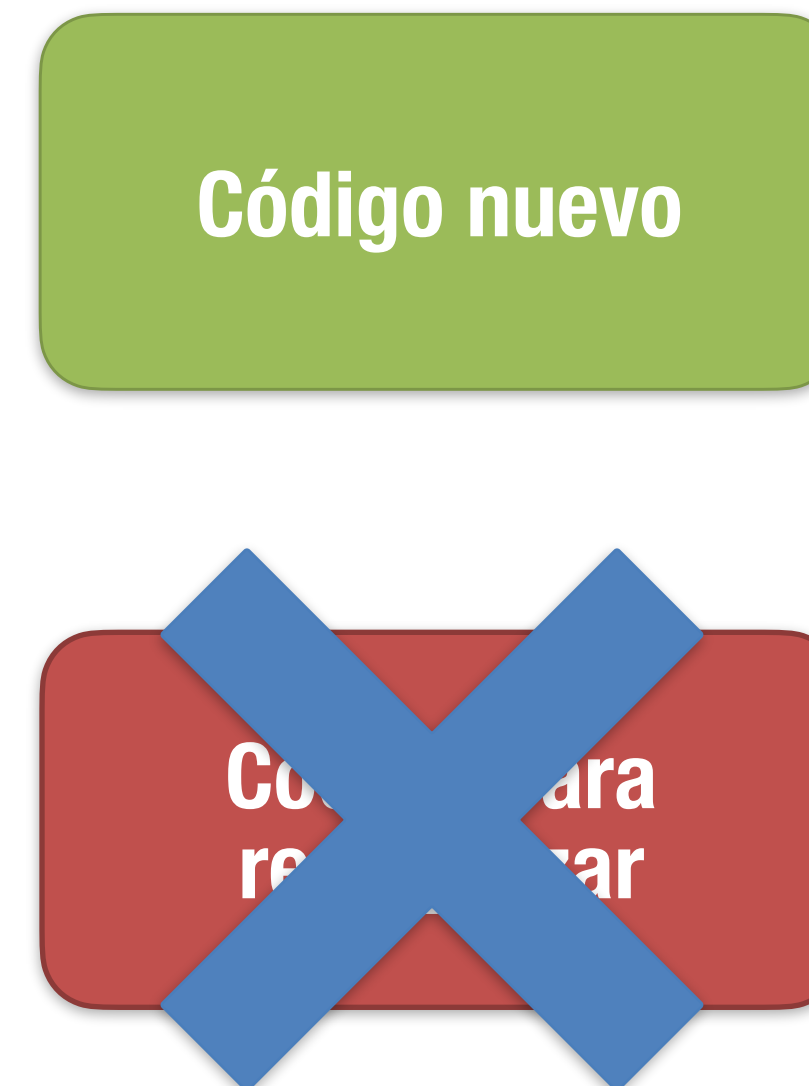
Aísla el código que quieres reemplazar en un método y/o cámbiale el nombre



Crea un nuevo método con el **mismo nombre y firma**. Llama al viejo desde el nuevo



Añade lógica antes o después de la llamada hasta dejar de necesitar el viejo (p.ej. *feature flag*)



Ejercicio

Queremos añadir soporte de log, validación, sanitización, plantillas, etc.

```
export class LegacyEmailService {
  sendEmail(to: string, subject: string, body: string): string {
    return `Email sent to ${to}, subject: ${subject}, body: ${body}`
  }
}

// Código de nuestra aplicación que usa directamente el servicio legacy
const legacyService = new LegacyEmailService()

export function notifyWelcome(userEmail: string): string {
  return legacyService.sendEmail(userEmail, 'Welcome!', 'Thanks for joining our app.')
}

export function notifyPasswordReset(userEmail: string): string {
  return legacyService.sendEmail(userEmail, 'Reset your password', 'Click the link to reset...')
}

export function notifyOrderConfirmation(userEmail: string, orderId: string): string {
  return legacyService.sendEmail(
    userEmail,
    'Order Confirmation',
    `Your order ${orderId} has been confirmed.`
  )
}
```


EJERCICIO FINAL

QuoteBot kata

<https://github.com/franiglesias/quotebot>

1. Introduce tests de caracterización en este código
2. Corrige los errores indicados en algunos comentarios
3. Identifica code-smells y arréglalos
4. Introduce una nueva política de cálculo de la propuesta basada en los meses transcurridos del año actual
5. Invierte e inyecta las dependencias

Notas:



- * Puedes hacer el ejercicio en cualquiera de los lenguajes propuestos
- * No puedes tocar el código en las carpetas **thirdparty** o **lib**

Refactoring Avanzado

por Fran Iglesias

Staff Software Engineer

